BÖLÜM 3: VERI MIMARISI — DURUMUN YAŞADIĞI, ÖLDÜĞÜ VE YALAN SÖYLEDIĞI YER 💣 9 dk • DERS 12

Parça 12: Önbellekleme Stratejileri: Bilgisayar Biliminin İki Zor Şeyinden Biri

Sistemi bayat veya hayalet verilerle zehirlemeden performansı 10 katına çıkarmanın yolları.

> 🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**
>
> Bir spor yayın platformu veritabanını rahatlatmak için kullanıcı abonelik yetkilerini Redis'te 24 saat önbelleğe aldı. Bir kullanıcı derbi maçını izlemek için 'Standart' paketten 'Süper Lig VIP' paketine yükseltme yaptı. Kartından anında 499 TL çekildi. Ancak maçı açmak istediğinde ekranda 'Bu yayını izlemek için paketinizi yükseltin' uyarısı çıktı. Tekrar satın aldı, kartından bir 499 TL daha çekildi! Maç boyunca 40.000 kişi aynı hatayı yaşayıp şirketi sosyal medyada linç etti. Neden mi? Satın alma servisi veritabanını güncellemişti ancak önbellek anahtarını temizleyen hiçbir kod yoktu; sistem 24 saatlik sürenin (TTL) dolmasını bekliyordu!

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Phil Karlton'ın efsanevi sözü mimarları yıllardır uyarır: 'Bilgisayar biliminde sadece iki zor problem vardır: Önbellek geçersiz kılma (cache invalidation) ve isimlendirme.' Önbellek sihirli bir hız butonu değildir; aynı gerçeğin iki kopyasını tutma yönünde açık bir karardır. Önbellek eklediğiniz anda tutarsızlık, bayat veri ve izdiham (stampede) risklerini kabul etmiş olursunuz.

Temel Mimari İlkeler ve Zihinsel Modeller

Temel Önbellekleme Modelleri

- **Cache-Aside (Tembel Yükleme):** Uygulama önce önbelleğe bakar. Yoksa veritabanından okur, önbelleğe yazar ve döner. En yaygın ve güvenli model.
- **Write-Through:** Uygulama önbelleğe yazar, önbellek katmanı anında veritabanına yazar. Her zaman tazedir, ancak yazma gecikmesi artar.
- **Write-Behind (Gecikmeli Yazma):** Uygulama önbelleğe yazar, önbellek arka planda asenkron olarak DB'ye aktarır. Çok hızlıdır ama Redis çökerse veri kaybı riski taşır.

Önbellek İzdihamı (Cache Stampede / Thundering Herd)

Saniyede 10.000 istek alan popüler bir anahtarın süresi tam 12:00:00'da dolduğunda, aynı milisaniyede 10.000 isteğin hepsi önbelleği boş bulup veritabanına hücum eder ve veritabanını anında çökertir. Çözüm: Dağıtık kilitler (SingleFlight) ve rastgele TTL esnetmesi (Jitter).

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Geçersiz Kılma Kancası Eksikliği:** Veritabanında satırı güncelleyip Redis'teki eski anahtarı silmeyi unutmak.
- **Sonsuz Önbellek:** TTL belirlemeden Redis'e veri basıp belleğin dolmasıyla sunucuyu OOM (Out of Memory) ile patlatmak.
- **İş Mantığını Önbelleğe Almak:** 10 farklı değişkene bağlı karmaşık hesaplama sonuçlarını önbelleğe alıp bir değişken değiştiğinde geçersiz kılmayı imkansız hale getirmek.

🎯 MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR

- **1. Adım:** İstisnasız her önbellek anahtarına bir TTL (Yaşam Süresi) tanımla.
- **2. Adım:** Yazma anında önbellekteki veriyi değiştirmek yerine doğrudan sil (`DEL key`). Bırakın bir sonraki okuma veritabanından tazesini getirsin.
- **3. Adım:** Bütün TTL'lere rastgele sapma (jitter) ekle: `TTL = 300sn + rand(0, 60sn)`. Böylece tüm anahtarlar aynı saniyede düşmez.

Mimari Çıkarımlar ve Hile Tablosu

- Önbellek bir performans hızlandırıcıdır, birincil veritabanı değildir. Önbellek silindiğinde sisteminiz çöküyorsa mimarisi bozuktur.
- Yazma anında önbelleği güncellemektense anahtarı silmek her zaman daha güvenlidir.
- İzdihamı önlemek için TTL sürelerine mutlaka rastgele varyasyon (jitter) ekleyin.

🎙️ UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU

Ders Değerlendirmesi ve Ödev: Redis veya Memcached kümenizi inceleyin: TTL tanımlanmamış anahtarlar var mı? Redis sunucunuz şu an aniden yeniden başlasa ana veritabanınız ayakta kalabilir mi?